

TalentLens

Intelligent Candidate Discovery & Ranking

Rank on meaning, not keywords - then discount by credibility and reachability.

India Runs on Data & AI · config-driven · embeddings-based · CPU-friendly

The dataset is a trap for keyword matchers

A “Marketing Manager” who pasted Pinecone / RAG / FAISS into skills is NOT a fit.

A real engineer who built a recommender at a product company - but never wrote “RAG” - IS a fit.

Dormant for 6 months with a 5% response rate? Not actually reachable - down-weight.

So we never score keywords. We score intent, then multiply by how much we can trust it.

One readable formula

$$\text{fit} = (0.55 \cdot \text{semantic} + 0.30 \cdot \text{skill_coverage} + 0.15 \cdot \text{seniority}) \times \text{credibility} \times \text{availability}$$

semantic

MiniLM cosine between the candidate's career and the JD's intent. Finds fits who skip the buzzwords.

skill_coverage

How many of the 4 must-have families (retrieval, vector search, ranking, evaluation) are evidenced.

seniority

Triangular fit: peaks at 6-8y, in-band 5-9y, soft outside.

credibility $\times 0.15-1.0$

Off-domain title + no eng history $\rightarrow \times 0.18$. "Expert" with 0 months $\rightarrow \times 0.6$. Services-only, fabricated tenure, LangChain-only $\rightarrow$ smaller hits.

availability $\times 0.55-1.0$

Last-active recency, recruiter-response rate, open-to-work. Down-weights the unreachable - gently.

The role lives in files, not code

`jobs/<role>/description.md` + `spec.json` → drop a new folder, rank a new role.

`description.md`

The full JD in plain Markdown. Embedded verbatim, so the match captures intent - “shipped retrieval at a product company, not a researcher or title-chaser.”

`spec.json`

Skill families, scoring weights, seniority band and the credibility rules (off-domain titles, services firms, framework-only). Tune the role without touching the engine.

Why multiplicative beats filtering

Hard filters silently drop people. We never drop - every candidate keeps a score and a reason.

A keyword-stuffer can have a high semantic score, but $\times 0.18$ credibility pins them to the bottom:

✓ **Recommendation Systems Engineer @ Swiggy - fit 1.00, 4/4 must-haves, active, 91% response**

⚠ **Marketing Manager @ Acme with FAISS/RAG in skills - fit ~ 0.00 , no eng role in career history**

Built to ship

compact & auditable

One engine file + a job folder. The score is four arithmetic terms you can verify by eye, with a reason per candidate.

embeddings, cached

sentence-transformers MiniLM, auto GPU/CPU. Encodes once and caches to disk; full 100K pool in ~2-3 min on a Colab GPU.

Docker + Colab

Reproducible Dockerfile and a one-click Colab that pulls the dataset from Drive and ranks every candidate.

Rank on meaning. Trust by evidence.

```
python rank.py --job jobs/senior_ai_engineer --candidates candidates.jsonl
```